Evaluating Prompt Engineering Methods for Edge Deployment of Small Language Models*

Ryan Hare, Yanlai Wu, Travis Lorsong, Vincent Mullary, Dr. Ying Tang

***Abstract*—In recent research and development, large language models have seen rapid iterations and improvements. However, there are still many limitations in place when applying these models to real-world applications such as connectivity requirements, API costs, and model response times. For edge applications in particular, smaller open-source models are often preferred due to their ability to run on local hardware and quick response times. But these models still have drawbacks in terms of response accuracy and quality. To that end, this work presents an evaluation of advanced prompting methods and configurations to determine the effectiveness of these methods on compact large language models. Specifically, evaluations focus on models' deductive reasoning and domain-specific knowledge within several benchmarks.**

I. Introduction

The rapid proliferation of Large Language Models (LLMs) has fundamentally transformed the landscape of natural language processing, artificial intelligence, and autonomous systems [1]. State-of-the-art models (for example, GPT-5.4 and Gemini 3.1) exhibit remarkable emergent abilities in terms of zero-shot reasoning and complex problem-solving [2]. However, their massive parameter counts can pose significant barriers to real-world deployment. These models require substantial computational resources, extensive memory overhead, and high-bandwidth cloud infrastructure [3], or alternatively, long-term API costs and constant network connectivity [4]. Consequently, these models are unsuitable for resource-constrained environments, edge computing, and privacy-sensitive local applications, which are increasingly central to cyber-physical systems. To address these bottlenecks, researchers and practitioners are increasingly turning to compact or \"small\" Language Models (SLMs) [5]. Typically ranging from 3 billion to 15 billion parameters, SLMs offer a more accessible, deployable, and cost-effective alternative for local AI applications [6].

Despite their advantages in cost and efficiency, SLMs often suffer from a noticeable performance gap compared to their larger counterparts when deployed out-of-the-box [5]. To bridge this gap, literature has heavily featured prompt engineering [7]. Techniques such as Few-Shot prompting [8], Chain-of-Thought (CoT) [9], Tree of Thoughts (ToT) [10], and self-reflection [11] have emerged as critical drivers for enabling complex reasoning and unlocking model capabilities independent of computationally expensive fine-tuning and exponential model parameter size increases. Furthermore, recent works in test- and inference-time optimization strategies have demonstrated augmentations to the inference steps that can improve a model's reasoning trajectory [12] such as inference scaling [13] and reasoning distillation [14] to achieve better results with SLMs.

Our eprimary motivation for this study stems from a critical gap in current AI deployment strategies: while structural prompt engineering greatly improves 100B+ parameter models, it remains uncertain how effectively these methods scale down to resource-constrained networks and SLMs. Existing works have evaluated SLMs for specific problems such as code generation [15] and cybersecurity [16], but no efforts have been made to evaluate SLMs on a general-purpose reasoning benchmark to show effective best practices for prompting in SLM applications, as well as providing a quantitative comparison of methods. The small size of SLMs, in particular, may prove to negate the benefits of structured prompting methods. For instance, a complex Tree of Thoughts, an effective method in larger models, may be unable to scale down properly to fit the capacity of a 4B parameter model. Furthermore, even if inference-time optimization such as inference scaling improves an SLM's accuracy, it is possible that the added response latency or computation time negates the benefits of applying an SLM in the first place.

To that end, the main contributions of this paper are as follows:

1) We present a comprehensive comparative analysis of multiple prompt engineering methodologies (CoT, ToT, Self-Reflection) tested on three varied open-source SLMs (Qwen 3.5 architectures, Trinity Large, Step 3.5 Flash).

2) We introduce a custom generation framework and benchmark evaluation scheme to evaluate SLM reasoning on typical LLM benchmark tasks.

3) We evaluate SLM latency, accuracy, and reasoning on benchmark datasets while applying a variety of methods of both prompt engineering and LLM response optimization to determine the impact of these methods on SLM applications.

II. Methodology

To rigorously evaluate reasoning in SLMs and address our core problem statement, we evaluate both structural prompting methods and inference optimization methods. Structural prompting methods are specially engineered prompts designed to tailor LLM output, encourage certain methods of thought, or otherwise assist the LLM in answering prompts logically. Inference optimization methods, meanwhile, are methods for improving LLM output that focus on the inference stage, often aiming to avoid hallucinations through, for instance, majority voting based on multiple responses to the same prompt. The specific methods applied with examples are detailed below.

A. Structural Prompting Methods

Using a standardized prompting framework, we apply a variety of common and effective prompt engineering methods, detailed below with examples.

1. Zero-Shot (Direct): Instructions strictly compel the model to answer directly with no intermediate explanation or reasoning chain. This serves as our baseline. Taking an example from one of the chosen benchmark data sets (discussed further below), an evaluation problem may take the form of shown below. Note that for this example, text-based symbols have been converted to words for human readability. For example, in the actual data set, a person X and animal y are noted to be in the same room with 'X @ y'. For more details on the notation used, see the dataset discussed in a later section.

```
There are 3 rooms with persons Alice, Bob, Charlie and pets anole, bat, cat. Each room contains exactly one person and one pet.

   Restrictions:

   1. Alice and anole are not in the same room.

   2. Bob is in a room before Alice

   3. cat is in room 3

   4. cat and anole are in adjacent rooms

   5. Charlie is not in room 1

   6. Bob and cat are not in adjacent rooms

What are the tenants in the rooms?
```

Figure 1: Zero-shot prompting often includes only a problem to be solved, no extra detail, consideration, or other tips given.

2. Chain-of-Thought (Multi-Step): Instructing the model to solve the puzzle systematically. We explicitly command the model to list all given constraints, deduce immediate implications, and resolve contradictions before deriving the final answer using a prompting structure shown below.

```
[Insert the same problem text from the zero-shot example]

Let's think step by step:

1) List all given constraints and interpret them using the legend.

2) Deduce immediate implications.

3) Explore possibilities and eliminate contradictions.

4) Construct the final solution.

Clearly state the final arrangement at the end using ONLY this format: Room 1: [Person], [Pet]…
```

3. Tree of Thoughts (ToT): Unlike Chain-of-Thought, ToT generates a "tree" of possible thought processes before backtracking and checking these processes against the requirements. Simulated via a persona prompt where the model roleplays a panel of experts: a Logic Expert analyzing hard constraints, a Scenario Expert generating hypothesis pairs, and a Reviewer checking for contradictions. So, for the same problem, the ToT prompt would be:

```
[Insert the same problem text from the zero-shot example]

Simulate a panel of experts solving this problem using the symbol legend:

- Logic Expert: Analyzes hard constraints.

- Scenario Expert: Tests potential assignments.

- Reviewer: Checks for contradictions.

Consensus Solution:

Clearly state the final arrangement at the end using ONLY this format: Room 1: [Person], [Pet]…
```

In this scenario, the SLM would then create multiple branches of the tree with different solutions given by the different experts before double-checking and agreeing on a consensus solution.

4. Critic (Self-Verify): A self-reflection framework where the model first proposes a solution, subsequently critiques its own reasoning against the predefined rule set, and refines the final answer. The same example will be solved using the self-verifying format below:

```
[Insert the same problem text from the zero-shot example]

First, solve the problem using the symbol legend.

Then, verify your solution against every constraint in the question.

Final Verified Solution:

Clearly state the final arrangement at the end using ONLY this format: Room 1: [Person], [Pet]…
```

5. Continuous Reasoning (COCONUT): Emphasizing consistency across multi-turn evaluations by maintaining and building upon sequential interactive memory instances. While the other methods are more direct prompt engineering, COCONUT uses a conversation memory to emulate a back-and-forth problem-solving conversation with an assistant, solving one "step" of the problem at a time before the user prompts to continue. Thus, the model is encouraged to focus on small aspects of the problem instead of the entire problem. In this case, the model would explicitly be told:

```
[Insert the same problem text from the zero-shot example]

Maintain consistent reasoning across turns and build on previous steps.

First, evaluate rule constraints regarding Bob and Alice.

<LLM RESPONSE>

Given prior facts, evaluate positions for Charlie and the cat.

<LLM RESPONSE>
```

```
<REPEAT UNTIL SOLUTION>
```

B. Inference Optimization Methods

In addition to structural prompting, we evaluate the efficacy of runtime inference optimization methods on SLMs:

- Base Execution: Standard single-pass generation.
- Inference Scaling (n-Voting): Generating n candidate responses (e.g., $n = 5$) per prompt. A majority heuristic then extracts the final variable and selects the best outcome to filter isolated hallucinations.
- Reasoning Distillation: A multi-agent configuration where a complex "Teacher" path (e.g., a Teacher executing ToT) evaluates the prompt, and its reasoning is compressed by a "Student" SLM to extract a final concise logic pattern.

III. Experimental Setup

A. Models for Evaluation

Our experiments focus on optimized, compact LLMs that are positioned for efficient inference. Selecting these models allows us to test the boundaries of "small" capacities. Specifically, we selected several models of varied size based on their ability to balance reasoning capability, inference efficiency, and practical usability. Within the 4B-27B parameter range, Qwen3.5-27B is chosen as a strong open-weight baseline model that could demonstrate competitive reasoning performance for its size while remaining computationally lightweight. In contrast, Qwen3.5-4B model offered a more aggressive optimization focused mainly on low-latency inference, allowing us to evaluate how highly optimized, speed-focused but still large models can perform on structured reasoning tasks. Together, these models served as a representation of two distinct design philosophies within small-scale LLMs with one prioritizing balanced reasoning capability and the other prioritizing inference speed and efficiency.

For the 100B–400B parameter class, Step3.5 Flash is selected since it offers a significant increase in model capacity and leverages MoE\cite{moe}, thus reducing memory footprint and improving throughput without sacrificing performance. Through these capabilities, the model is an ideal candidate for studying how scaling affects reasoning using the same hardware. As for Trinity-Large-Preview, the model is chosen to represent a reasoning-optimized model within the size range where it is designed to handle complex, multi-step tasks with larger context windows. These capabilities allow us to evaluate whether specialized reasoning-focused architectures provide any measurable advantages over general-purpose models of similar scale. To summarize, our evaluations used the following LLMs:

- 4B–27B Parameter Models: `Qwen3.5-4B` and `Qwen3.5-27B` represent fast, highly optimized small-to-medium scale models targeted for immediate inference.
- 100B–400B Parameter Class: `Step3.5 Flash` and `Trinity-Large-Preview` represent intermediate models capable of holding longer conversations while still running on reasonable local architectures.

Smaller and larger alternatives were considered but ultimately excluded to maintain a more focused and controller comparison. Models below 2B parameters were not included due to their consistently weak performance on multi-step reasoning tasks, significantly limiting any meaningful evaluation. Conversely, larger models, such as 70B+ or modern systems such as ChatGPT or DeepSeek, were excluded due to their substantially higher inference cost, reduced accessibility, and difficulty in isolating improvements attributable to reasoning strategies rather than the scale itself. In addition, these larger models had pre-existing data regarding these reasoning methods and results would not serve as meaningful data. Using these models, we were able to conduct a target investigation in how model size, optimization strategies, and reasoning techniques interact within a practical range of LLMs.

B. Testing Environment

For the SLM evaluation, all models were hosted locally on an Ubuntu operating system utilizing the SGLang \cite{sglang}. Hardware deployments consisted of a single NVIDIA GeForce RTX 4090 for the Qwen3.5 4B model and a single NVIDIA RTX 6000 Ada Generation GPU for the larger Qwen3.5 27B variant. To standardize the inference environment, SGLang deployment is configured with a tensor parallel size of 1, a static memory fraction of 0.55, a 256k max context length, and a chunked prefill size of 512.

Critically, SLMs attempting advanced CoT paths frequently succumb to infinite generation loops due to their lower attention span bounds. To address this, we implemented sliding-window n-gram tracking. If duplicated character blocks cross a 50% duplication threshold, or if stalled phrases exceed responses, our test environment terminates generation, logs the failure, and recursively reruns the test.

C. Benchmark Datasets

To evaluate SLM reasoning, we utilized two existing benchmark datasets determined from literature search. In choosing these two datasets, we aimed to find complex problems that require a certain level of reasoning to determine a solution.

The first dataset utilized is the *AI2_ARC* dataset [19]. This dataset features grade-school level science questions with multiple-choice answers. This dataset is designed to test LLMs on their advanced question-answering ability and on their inherent knowledge, while still requiring logical reasoning to determine the best response on more esoteric questions.

Second is the *Deductive Logical Reasoning – Room Assignment* dataset [20]. This set provides logical problems with symbolic relationships where some amount of persons and pets are placed within some amount of rooms. A set of logical rules limit the solution space, with a few examples shown below:

- Person X's room is before Person Y's room.
- Person X's room is next to Person Y's room, independent of order.

- Person X is assigned to Room N.
- Person X and Pet x are assigned to the same room.

The SLM response, then, is to simply give the following, where (X, Y, Z) represent people and (x, y, z) represent pets:

- Room 1: X, x
- Room 2: Y, y
- Room 3: Z, z

The problems are infinitely scalable by adding more rooms, people, pets, and logical rules, while also guaranteeing that at a single correct solution exists.

IV. Conclusions

Conclusions, importance of this work.

Appendix

Appendixes should appear before the acknowledgment.

Acknowledgment

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression, "One of us (R. B. G.) thanks . . ." Instead, try "R. B. G. thanks". Put sponsor acknowledgments in the unnumbered footnote on the first page.

References

[1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," *arXiv*, Mar. 2023. doi: 10.48550/arXiv.2303.18223.

[2] L. Berti, F. Giorgi, and G. Kasneci, "Emergent abilities in large language models: A survey," *arXiv*, Feb. 2025. doi: 10.48550/arXiv.2503.05788.

[3] Z. Gu, H. Ye, X. Chen, Z. Zhou, H. Feng, and Y. Xiao, "StrucText-Eval: Evaluating large language model's reasoning ability in structure-rich text," in *Proc. 63rd Annu. Meet. Assoc. Comput. Linguistics (ACL)*, Vienna, Austria, Jul. 2025. [Online]. Available: https://aclanthology.org/2025.acl-long.1563/

[4] A. N. Author et al., "(Metadata needed) IEEE Xplore document 10685415," *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10685415

[5] S. Subramanian, V. Elango, and M. Gungor, "Small language models (SLMs) can still pack a punch: A survey," *arXiv*, Jan. 2025. doi: 10.48550/arXiv.2501.05465.

[6] A. N. Author et al., "(Metadata needed) IEEE Xplore document 10590016," *IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10590016

[7] B. Chen, Z. Zhang, N. Langrené, S. Zhu, et al., "Unleashing the potential of prompt engineering for large language models," *Patterns*, vol. 6, no. 6, p. 101260, 2025. doi: 10.1016/j.patter.2025.101260.

[8] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33 (NeurIPS 2020). [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35 (NeurIPS 2022). [Online]. Available: https://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[10] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, vol. 36 (NeurIPS 2023). [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html

[11] M. Renze and E. Guven, "Self-reflection in LLM agents: Effects on problem-solving performance," *arXiv*, May 2024. doi: 10.48550/arXiv.2405.06682.

[12] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv*, Aug. 2024. doi: 10.48550/arXiv.2408.03314.

[13] Z. Hou, X. Lv, R. Lu, J. Zhang, Y. Li, Z. Yao, J. Li, J. Tang, and Y. Dong, "T1: Advancing language model reasoning through reinforcement learning and inference scaling," *arXiv*, Jan. 2025. doi: 10.48550/arXiv.2501.11651.

[14] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, "Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes," *arXiv*, May 2023. doi: 10.48550/arXiv.2305.02301.

[15] M. A. Al Mamun, S. Nath, G. Uddin, and N. Deb, "Evaluating the environmental impact of using SLMs and prompt engineering for code generation," *arXiv*, Apr. 2026. doi: 10.48550/arXiv.2604.02776.

[16] G. Almeida, M. Pohlmann, A. Severo, D. Kreutz, T. Heinrich, and L. Pereira, "On-premise SLMs vs. commercial LLMs: Prompt engineering and incident classification in SOCs and CSIRTs," *arXiv*, Nov. 2025. doi: 10.48550/arXiv.2511.14908.

[17] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[18] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," *arXiv*, Dec. 2023. doi: 10.48550/arXiv.2312.07104.

[19] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and Ø. Tafjord, "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," *arXiv*, Mar. 2018. doi: 10.48550/arXiv.1803.05457.

[20] E. Munah, "Deductive logical reasoning – room assignment (dataset)," Hugging Face Datasets, 2024. [Online]. Available: https://huggingface.co/datasets/emunah/deductive_logical_reasoning-room_assignment